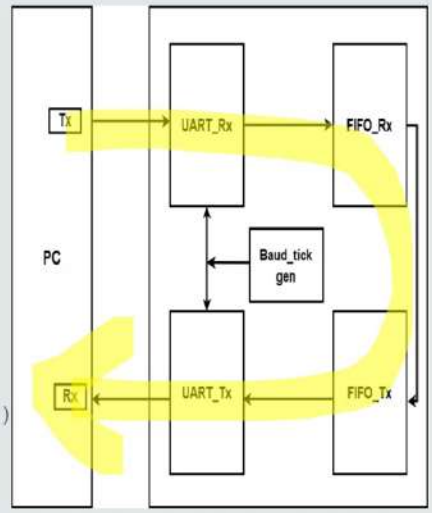
모듈 구조

- 기존 UART Loop back은 PC - UART Rx - UART Tx - PC 흐름으로 진행되었고
  UART Rx - UART Tx사이에 FIFO를 추가하여 약간의 지연을 발생시킨 후 다시 PC로 반환

- **Baud tick**
  - baud rate 인 9600bps 간격의 tick 생성기
    (Rx, Tx 모두에 사용하기 위해 1/16으로 추가 분주됨)

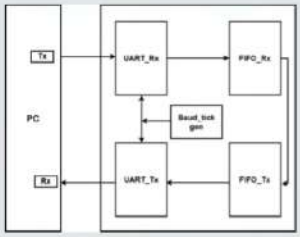
- **UART Tx, Rx**
  - 외부로부터 데이터를 baud rate에 맞춰 송, 수신하는 모듈

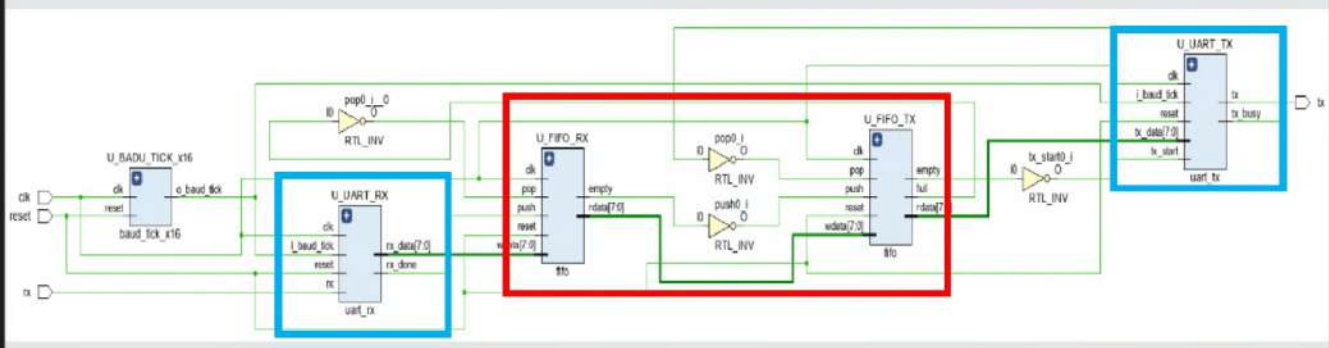
- **FIFO**
  - FIFO는 데이터를 임시 저장하고 다시 들어온 순서대로 출력하는 모듈(주로 버퍼로 이용됨)
  - 현재 시뮬레이션에선 UART RX - UART TX 사이에 적용되어
    Rx된 데이터가 바로 Tx 되지 않고 지연을 주기 위함



RTL Schematic

- 입력이 UART Rx를 시작으로 FIFO Rx, FIFO Tx, UART Tx를 통해 출력되는 흐름





Free trial

chang-hyung Kim(발표 중, 주석 다는 중)

코드구현 - FIFO

<Register_file>

```
module register_file #(
    parameter WIDTH = 2
) (
    input           clk,
    input  [WIDTH-1:0] waddr,
    input  [WIDTH-1:0] raddr,
    input  [     7:0] wdata,
    input           we,
    output [     7:0] rdata
);
    parameter DEPTH = 2 ** WIDTH;
    reg [7:0] register_file[0:DEPTH-1];
    // rdata : CL output


    always @(posedge clk) begin
        if (we) begin
            register_file[waddr] <= wdata;
        end
    end

    // assign rdata = (!we) ? register_file[raddr] : 8'hz;
    assign rdata = register_file[raddr];
endmodule
```

- 데이터를 저장하고 출력하는 RAM
- RAM의 쓰기 주소w_addr, 읽기 주소 r_addr로 구분
- WE여부에 따라 w_adder 주소에 입력 데이터(wdata)를 저장
- 출력은 조합으로 구성되어 r_adder 위치의 데이터 상시 출력

<Control>

```
// state register
always @(posedge clk, posedge reset) begin
    if (reset) begin
        wptr_reg  <= 0;
        rptr_reg  <= 0;
        full_reg  <= 0;
        empty_reg <= 1;
    end else begin
        wptr_reg  <= wptr_next;
        rptr_reg  <= rptr_next;
        full_reg  <= full_next;
        empty_reg <= empty_next;
    end
end
```

```
// next state CL
always @(*) begin
    wptr_next  = wptr_reg;
    rptr_next  = rptr_reg;
    full_next  = full_reg;
    empty_next = empty_reg;

    case ({
        push, pop
    })
        2'b00: begin
        end

        2'b01: begin  // pop only
            rptr_next = rptr_reg + 1;
            full_next = 0;
            if (wptr_reg == rptr_next) empty_next = 1;
        end

        2'b10: begin  // push only
            wptr_next  = wptr_reg + 1;
            empty_next = 0;
            if (wptr_next == rptr_reg) full_next = 1;
        end

        2'b11: begin  // push_pop
            if (full) begin
                rptr_next = rptr_reg + 1;
                full_next = 0;
            end else if (empty) begin
                wptr_next  = wptr_reg + 1;
                empty_next = 0;
            end else begin
                wptr_next = wptr_reg + 1;
                rptr_next = rptr_reg + 1;
            end
        end
    endcase
end
```

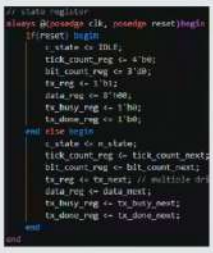
- RAM의 읽기 쓰기를 제어
- 외부에서 push 신호가 오면 읽기, pop 신호가 오면 쓰기를 진행
- 각 상태에 알맞게 읽기, 쓰기 동작이 이루어지게 신호를 RF에 전송
  - wptr : 데이터를 쓸 주소
  - rptr : 데이터를 읽을 주소
  - full : 모든 메모리가 가득 참을 알림
  - empty : 모든 메모리가 비었음을 알림

나 동민
This appears to be a long document. Save time by reading a summary using AI Assistant.
Free trial
chang-hyung Kim(발표 중, 주석 다는 중)

코드구현 - UART_Tx

- FSM에 의한 제어 (IDLE, START, DATA, STOP)
- 입력 데이터가 들어오면 baud rate로 한 bit씩 출력

- 주요 신호 설명
  - tick_count : baud rate 의 한 주기 중 어디인지 가리킴
  - bit_count : 8번의 data bit 전송 확인
  - tx : 데이터가 1bit씩 출력되는 라인
  - data : 입력 tx_data와 연결됨
  - tx_busy : bit 전송 상태임을 알림
  - tx_done : 데이터 전송 후 STOP -> IDLE로 상태 변경 알림



```
// state register
always @(posedge clk, posedge reset)begin
    if(reset) begin
        c_state <= IDLE;
        tick_count_reg <= 4'b0;
        bit_count_reg <= 3'b0;
        tx_reg <= 1'b1;
        data_reg <= 8'h00;
        tx_busy_reg <= 1'b0;
        tx_done_reg <= 1'b0;
    end else begin
        c_state <= n_state;
        tick_count_reg <= tick_count_next;
        bit_count_reg <= bit_count_next;
        tx_reg <= tx_next; // multiple dri
        data_reg <= data_next;
        tx_busy_reg <= tx_busy_next;
        tx_done_reg <= tx_done_next;
    end
end
```

```
IDLE: begin
    tick_count_next = 0;
    // output
    tx_next = 1'b1;
    tx_done_next = 0;
    tx_busy_next = 0;
    // condition of next transition
    if(tx_start)begin
        data_next = tx_data;
        n_state = START;
        tx_busy_next = 1;
    end
end

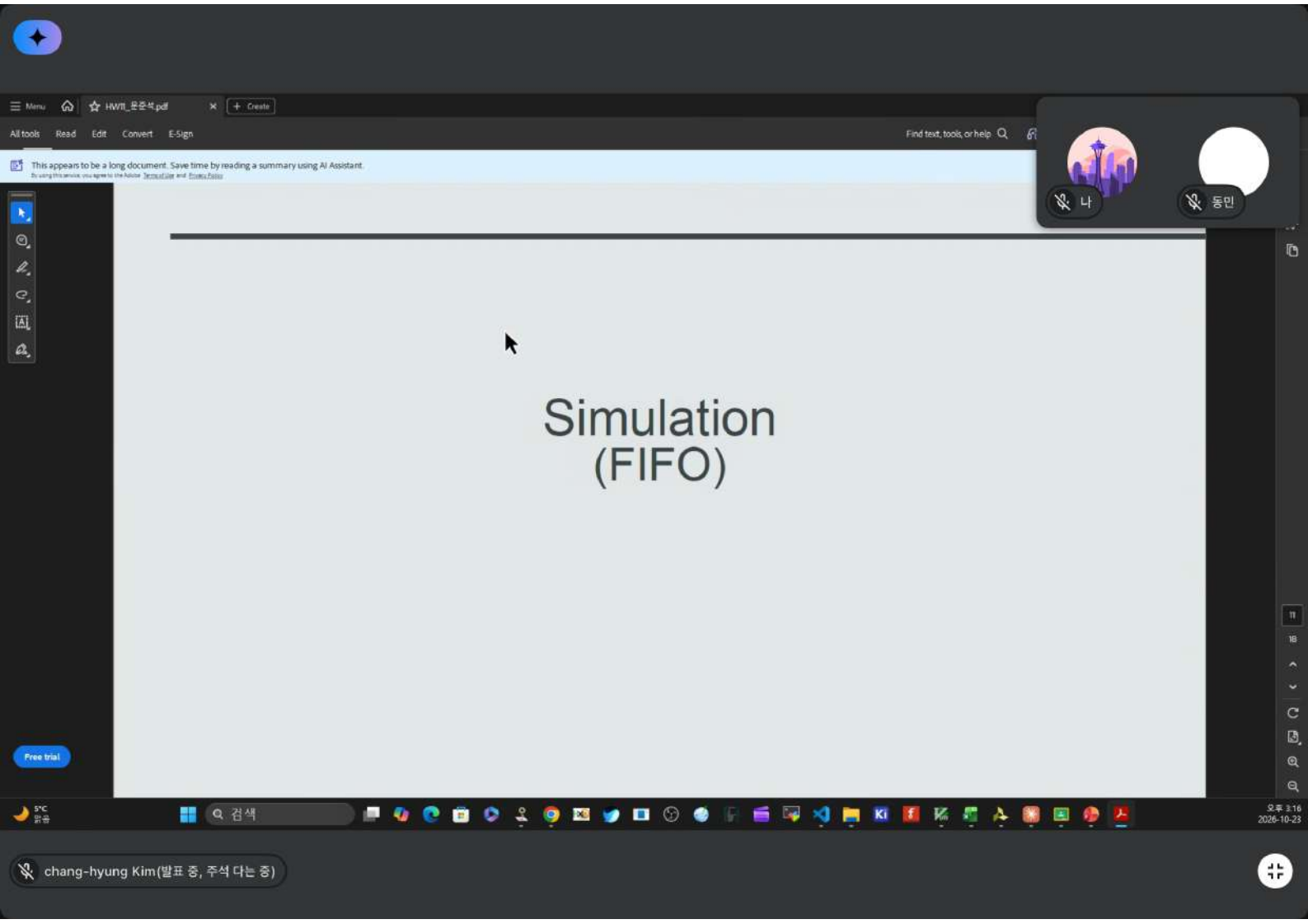
START: begin
    // output
    tx_next = 1'b0;
    bit_count_next = 3'd0;
    if(i_baud_tick) begin
        if(tick_count_reg == 15) begin
            tick_count_next = 0;
            n_state = DATA;
        end else begin
            tick_count_next = tick_count_reg + 1;
        end
    end
end
```

```
DATA: begin
    tx_next = data_reg[0];
    if(i_baud_tick)begin
        if(tick_count_reg == 15)begin
            //data_next = data_reg >> 1; // 이 방법도 가능
            data_next = {1'b0, data_reg[7:1]};
            tick_count_next = 0;
            if(bit_count_reg == 7) begin
                n_state = STOP;
            end else begin
                n_state = DATA;
                bit_count_next = bit_count_reg + 1;
            end
        end else begin
            tick_count_next = tick_count_reg + 1;
        end
    end
end

STOP: begin
    tx_next = 1'b1;
    if(i_baud_tick)begin
        if(tick_count_reg == 15) begin
            tx_busy_next = 0;
            tx_done_next = 1;
            n_state = IDLE;
        end else begin
            tick_count_next = tick_count_reg + 1;
        end
    end
end
```

나 동민

chang-hyung Kim(발표 중, 주석 다는 중)


Menu
HW11_윤은석.pdf
Create
All tools
Read
Edit
Convert
E-Sign
Find text, tools, or help
This appears to be a long document. Save time by reading a summary using AI Assistant.
나
동민
Simulation
(FIFO)
Free trial
검색
chang-hyung Kim(발표 중, 주석 다는 중)

Simulation source (FIFO)

- For문을 통해 총 256번의 랜덤 상태 테스트

- 랜덤 입력 값을 생성
- FIFO의 작동 조건인 posedge clk에 데이터를 인가하지 않고 1ns만큼 뒤에 랜덤 입력 값을 drive함
  (drive에 어떤 값이 만들어 졌는지 즉시 display)

- 2ns만큼 딜레이 하여 posedge clk + 3ns 지점에서 full 과 empty 상태를 체크함
  - full ,empty 신호가 떴을 때 Push cnt, pop cnt가 동일하다면 정상적인 상태로 pass cnt 증가
  - full, empty의 판별 조건은 push cnt, pop cnt가 일치하는 것으로 두 구문을 합칠 수도 있지만 편의를 위해 분리
  - full, empty가 동시에 발생할 수 없으므로 고려하지 않음

```
for (i = 0; i < 256; i = i + 1) begin
    @(posedge clk);
    #1;
    push  = $random % 2;
    pop   = $random % 2;
    wdata = $random % 256;
    // strobe로 하면 상황에 따라 섞이는 경우가 존재
    $display("%t : push = %d, pop = %d, wdta = %d", $time, push, pop,
             wdata);
```

```
#2;
if(full) begin
    if(push_cnt == pop_cnt) begin
        $display("%t : full is correct! push_cnt = %d, pop_cnt = %d", $time, push_cnt, pop_cnt);
        full_pass_cnt = full_pass_cnt + 1;
    end else begin
        $display("%t : full is not correct! push_cnt = %d, pop_cnt = %d", $time, push_cnt, pop_cnt);
        full_fail_cnt = full_fail_cnt + 1;
    end
end

if(empty) begin
    if(push_cnt == pop_cnt) begin
        $display("%t : empty is correct! push_cnt = %d, pop_cnt = %d", $time, push_cnt, pop_cnt);
        empty_pass_cnt = empty_pass_cnt + 1;
    end else begin
        $display("%t : empty is not correct! push_cnt = %d, pop_cnt = %d", $time, push_cnt, pop_cnt);
        empty_fail_cnt = empty_fail_cnt + 1;
    end
end
```

나 동민
This appears to be a long document. Save time by reading a summary using AI Assistant.
Free trial
chang-hyung Kim(발표 중, 주석 다는 중)

Simulation source (FIFO) (Cont'd)

- 출력 값을 읽는 지점은 negedge clk로 맞추어 meta stability 문제 방지
- 조합 출력이기에 for문 흐름 상 이전 입력 값들을 검증하는 구조로 구현됨
- Full이 아닌 상태에서 push가 일어나면 현재 push 주소가 가리키는 buffer에 값을 넣고 포인터 1 증가
- Empty 가 아닌 상태에 pop이 일어나면 현재 pop 주소가 가리키는 buffer의 값을 꺼내서 현재 rdata출력과 일치하는지 확인
- Display는 해당 위치가 즉시 실행되기에 cnt의 갱신 이전에 위치함

```
// 여기서 검증하는 것은 이전 사이클에 생성된 랜덤값을 검증하는 것
@(negedge clk);
if (!full & push) begin
    compare_buffer[push_cnt] = wdata;
    $display("%t : compaer_buffer = %d, push = %d", $time,
             compare_buffer[push_cnt], push);
    push_cnt = push_cnt + 1;
end

if (!empty & pop) begin
    if (compare_buffer[pop_cnt] == rdata) begin
        $display(
            "%t : PASS!! compare_data = %d, rdata = %d, pop = %d, empty = %d",
            $time, compare_buffer[pop_cnt], rdata, pop, empty);
        pass_cnt = pass_cnt + 1;
    end else begin
        $display(
            "%t : FAIL!! compare_data = %d, rdata = %d, pop = %d, empty = %d",
            $time, compare_buffer[pop_cnt], rdata, pop, empty);
        fail_cnt = fail_cnt + 1;
    end
    pop_cnt = pop_cnt + 1;
end
end
```

- 최종 pass ,fail 카운트 수를 표시하기 위한 display 표시

```
$display("%t : pass count = %d, fail count = %d", $time, pass_cnt, fail_cnt);
$display("%t : full pass count = %d, full fail count = %d", $time, full_pass_cnt, full_fail_cnt);
$display("%t : empty pass count = %d, empty fail count = %d", $time, empty_pass_cnt, empty_fail_cnt);
```

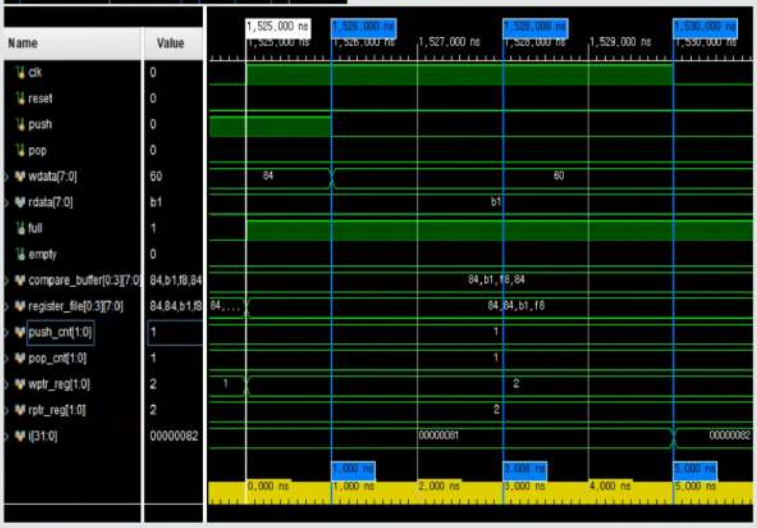
나 동민
chang-hyung Kim(발표 중, 주석 다는 중)

Simulation (FIFO)

- 1516ns에 push로 값을 입력으로 넣음
- 다가오는 상승엣지 clk 인 1525ns에 모듈 작동
- 1526ns에 다음 테스트 값 미리 입력
- 1528ns에 최근 상승엣지에 작동된 값에 대해 full, empty 검증 (full신호 발생시 push, pop cnt가 일치)
- 1530ns에 push, pop 동작 검증(full이기에 무시)



```
1516000 : push = 1, pop = 0, wdta = 132
1520000 : compaer_buffer = 132, push = 1
1526000 : push = 0, pop = 0, wdta =  96
1528000 : full is correct! push_cnt = 1, pop_cnt = 1
```

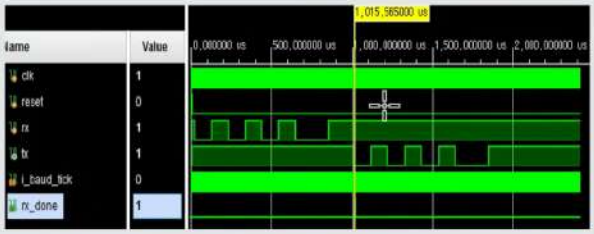


Simulation overview (FIFO)

- 전체적인 데이터 입, 출력 확인

- 데이터 0x95(0b1001_0101)을 Rx로 입력
  (역방향으로 start bit(0) -1-0-1-0-1-0-0-1- stop bit(1) 입력 시킴)
- Rx에서 모든 데이터를 받고 rx_done 신호가 발생하면
  그 신호를 FIFO_Rx의 push 신호로 인지하여 데이터가 넘어감
- 이후 FIFO_Tx, UART_Tx를 거쳐 최종 tx출력이 발생됨

- 시뮬레이션에선 **rx_done**이 발생되는 시점부터 **FIFO_Rx**로
  입력 및 출력, **FIFO_Tx**로 입력 및 출력 되는 과정을 살펴 볼 예정





Simulation (FIFO)

- rx_done 이후 FIFO Rx, Tx로 이어지는 흐름

- rx_done을 FIFO_Rx의 push 신호로 이용하여 0x95를 FIFO_Rx RAM에 저장 (다가오는 posedge clk에 저장됨)
- 이때 FIFO_Rx의 pop신호는 지속해서 인가되기에 push와 동시에 pop이 진행되어 FIFO_Tx로 값이 전달됨
- FIFO_Rx에서 push 후 empty 신호가 내려가는 순간 FIFO_Tx에선 push 신호로 인지하여 데이터를 쓰게됨
- 마찬가지로 FIFO_Tx관점에서의 UART_Tx의 Tx_start, Tx_busy와 신호가 연결되어 있어서 FIFO_Tx에 데이터가 차는 즉시 pop이 발생

- 정리하자면 rx_done이 뜨고 2개의 클럭 뒤에 데이터가 UART_Tx에 도달 (2개의 클럭 동기 FIFO를 사용하였음을 시사하는 바임)

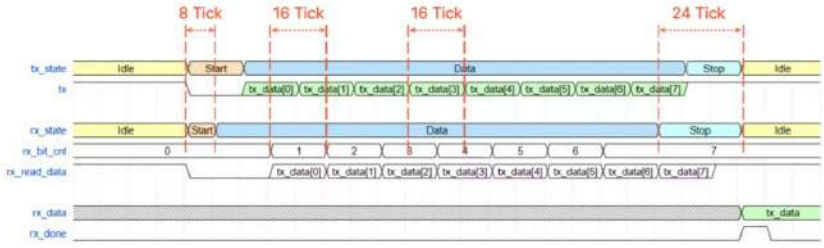


나
동민
chang-hyung Kim(발표 중, 주석 다는 중)

설계 이론

UART RX

- 구현을 위해 TX 및 RX를 동작시키는 Baud Tick Generator의 주파수를 Baudrate * 16으로 설정
- TX의 경우 Tick 입력 16개에 따라 동작을 수행
- RX의 경우 Start State를 Tick 입력 8개 이후 다음 State로 전이
- TX와 RX의 Idle State를 맞추기 위해 RX의 Stop에서 Tick 입력 24개 이후 다음 State로 전이



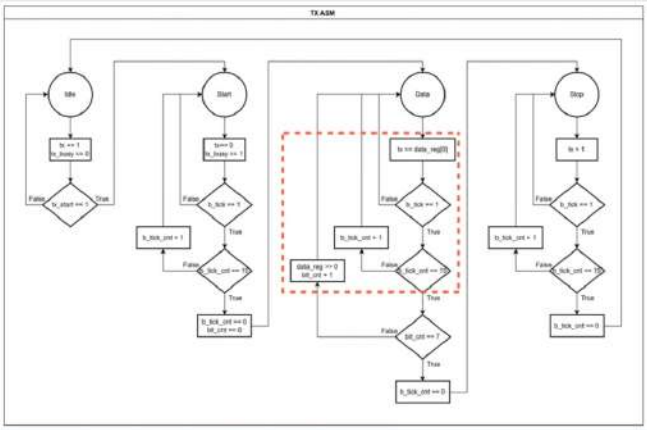
여기에 슬라이드 노트의 내용을 입력하십시오

chang-hyung Kim(발표 중, 주석 다는 중)

Algorithmic State Machine

TX ASM

TX 모듈 ASM

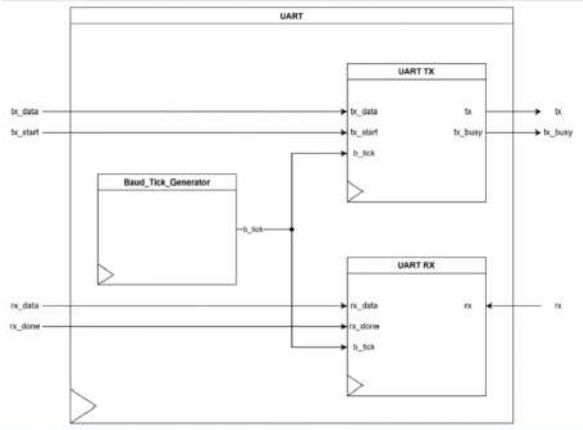


여기에 슬라이드 노트의 내용을 입력하십시오

chang-hyung Kim(발표 중, 주석 다는 중)

Block Diagram

UART Block Diagram



UART Block Diagram

1. Baud Tick Generator
   - UART 통신을 위한 Baudrate * 16 주파수의 Tick신호 생성

여기에 슬라이드 노트의 내용을 입력하십시오

chang-hyung Kim(발표 중, 주석 다는 중)

Simulation

UART Loopback RX

State	State Number
Idle	0
Start	1
Data	2
Stop	3

- 전송 데이터 : 00110000

1. 송신 값의 중앙에서 Read 동작 수행 확인
2. Stop State에서 rx_data로 값을 출력하는 것 확인



여기에 슬라이드 노트의 내용을 입력하십시오

chang-hyung Kim(발표 중, 주석 다는 중)

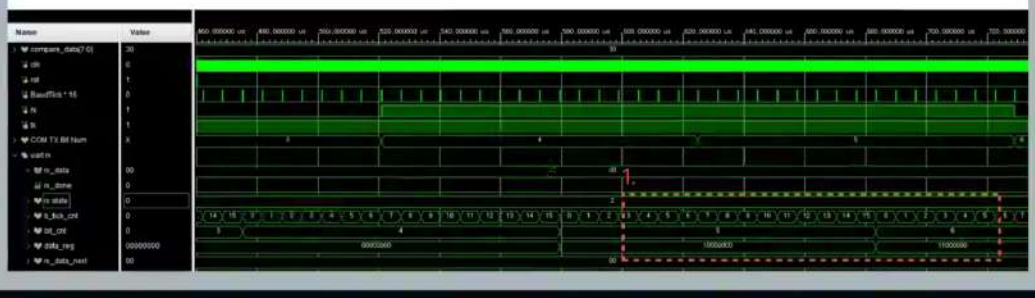
Simulation

UART Loopback RX

State	State Number
Idle	0
Start	1
Data	2
Stop	3

- 전송 데이터 : 00110000

1. bit_cnt가 증가하며 RX 입력 값을 읽어오는 것 확인
2. 입력 값 1을 MSB에 두고, Shift 연산 수행



여기에 슬라이드 노트의 내용을 입력하십시오

chang-hyung Kim(발표 중, 주석 다는 중)

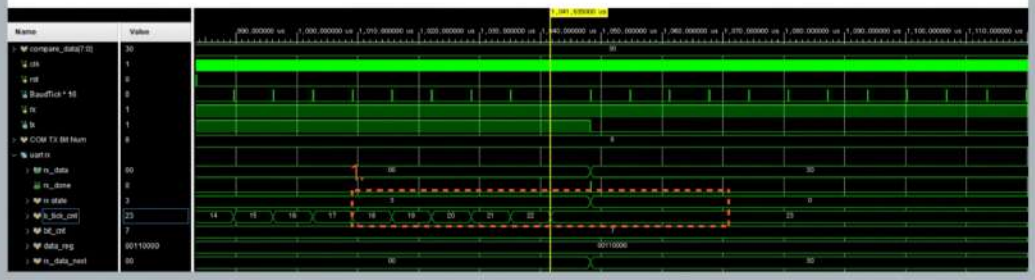
Simulation

UART Loopback RX

State	State Number
Idle	0
Start	1
Data	2
Stop	3

- 전송 데이터 : 00110000

1. 24개의 Tick 입력 이후 Stop 상태에서 Idle 상태로 전이



여기에 슬라이드 노트의 내용을 입력하십시오

chang-hyung Kim(발표 중, 주석 다는 중)

Simulation

UART Loopback TX

State	State Number
Idle	0
Start	1
Data	2
Stop	3

- 전송 데이터 : 00110000

1. tx_start 입력에 따라 수신 시작
2. RX로 부터 수신된 00110000 데이터 송신 확인



여기에 슬라이드 노트의 내용을 입력하십시오

chang-hyung Kim(발표 중, 주석 다는 중)

Simulation

UART Loopback TX

State	State Number
Idle	0
Start	1
Data	2
Stop	3

- 전송 데이터 : 00110000

1. Buad Tick 16개 입력에 대해 정보 전송 확인 및 Shift 연산 확인



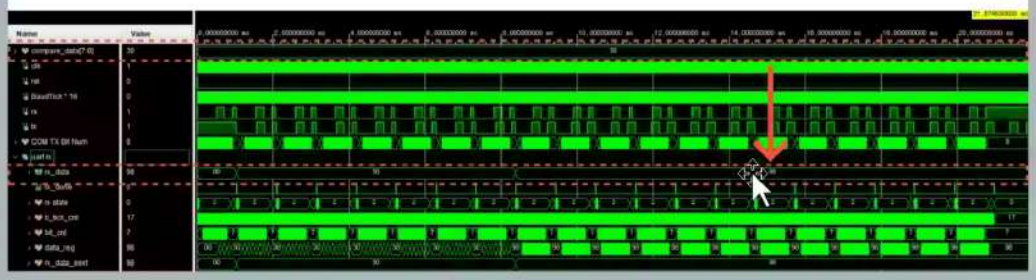
여기에 슬라이드 노트의 내용을 입력하십시오

chang-hyung Kim(발표 중, 주석 다는 중)

Trouble Shooting

UART RX

1. 동일한 입력에 대해 값이 비정상적으로 출력되는 것 확인



여기에 슬라이드 노트의 내용을 입력하십시오

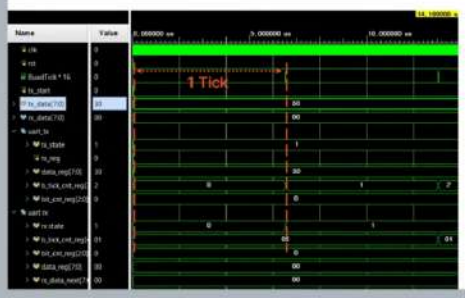
chang-hyung Kim(발표 중, 주석 다는 중)

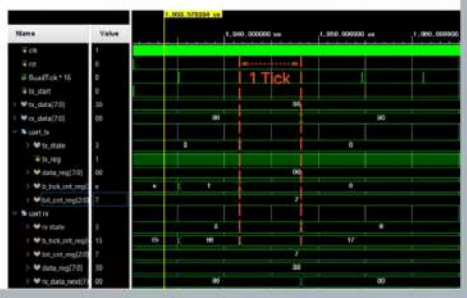
Trouble Shooting

UART RX

State	State Number
Idle	0
Start	1
Data	2
Stop	3

1. TX와 RX에서 Start 상태로 전이하는 시간이 1 Tick 차이나는 것을 확인
2. Stop에서 Idle 상태로 전이하는 시간차 역시 1 Tick 차이 확인





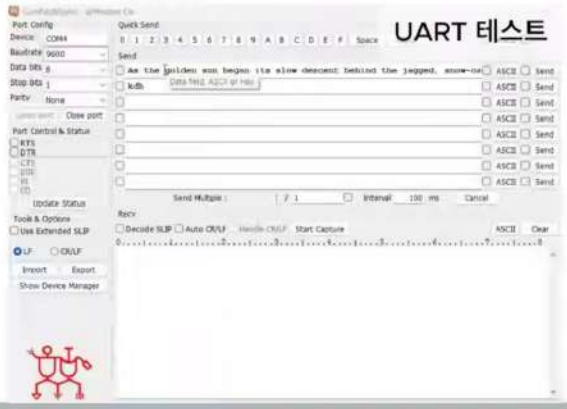
여기에 슬라이드 노트의 내용을 입력하십시오

chang-hyung Kim(발표 중, 주석 다는 중)

FPGA Implementation

UART Loopback

- 컴퓨터 송신 데이터가 수신되는 것을 확인



여기에 슬라이드 노트의 내용을 입력하십시오

chang-hyung Kim(발표 중, 주석 다는 중)

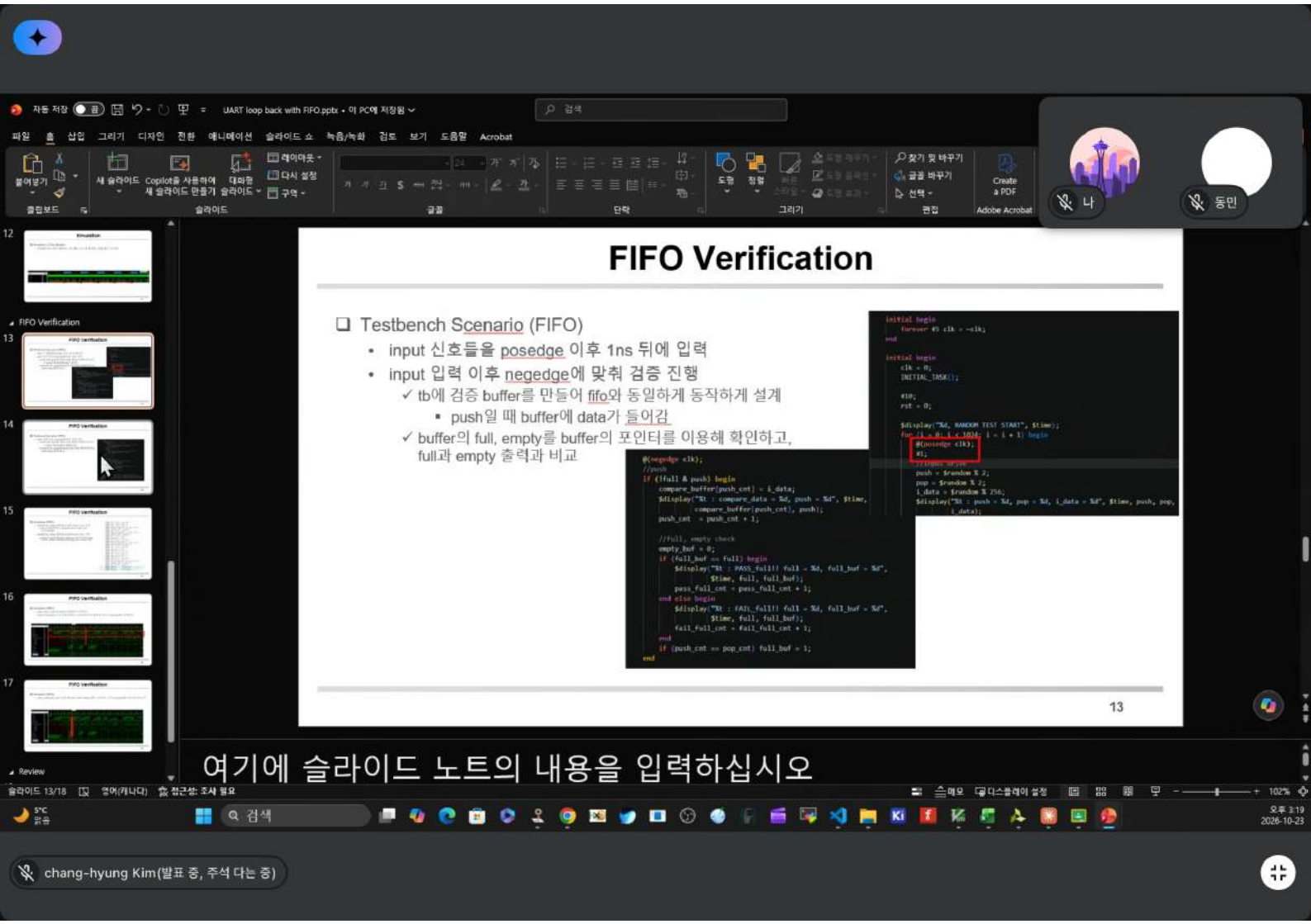
FIFO Verification

- Testbench Scenario (FIFO)
  - input 입력 이후 negedge에 맞춰 검증 진행
    - ✓ tb에 검증 buffer를 만들어 fifo와 동일하게 동작하게 설계
      - pop일 때 buffer에서 data를 꺼냄
    - ✓ buffer의 full, empty를 buffer의 포인터를 이용해 확인하고, full과 empty 출력과 비교



여기에 슬라이드 노트의 내용을 입력하십시오

chang-hyung Kim(발표 중, 주석 다는 중)


UART loop back with FIFO.pptx
나
동민
FIFO Verification
Testbench Scenario (FIFO)
input 신호들을 posedge 이후 1ns 뒤에 입력
input 입력 이후 negedge에 맞춰 검증 진행
tb에 검증 buffer를 만들어 fifo와 동일하게 동작하게 설계
push일 때 buffer에 data가 들어감
buffer의 full, empty를 buffer의 포인터를 이용해 확인하고, full과 empty 출력과 비교
13
여기에 슬라이드 노트의 내용을 입력하십시오
chang-hyung Kim(발표 중, 주석 다는 중)

FIFO Verification

- Testbench Scenario (FIFO)
  - input 입력 이후 negedge에 맞춰 검증 진행
    - ✓ tb에 검증 buffer를 만들어 fifo와 동일하게 동작하게 설계
      - pop일 때 buffer에서 data를 꺼냄
    - ✓ buffer의 full, empty를 buffer의 포인터를 이용해 확인하고, full과 empty 출력과 비교



14

여기에 슬라이드 노트의 내용을 입력하십시오

chang-hyung Kim(발표 중, 주석 다는 중)

FIFO Verification

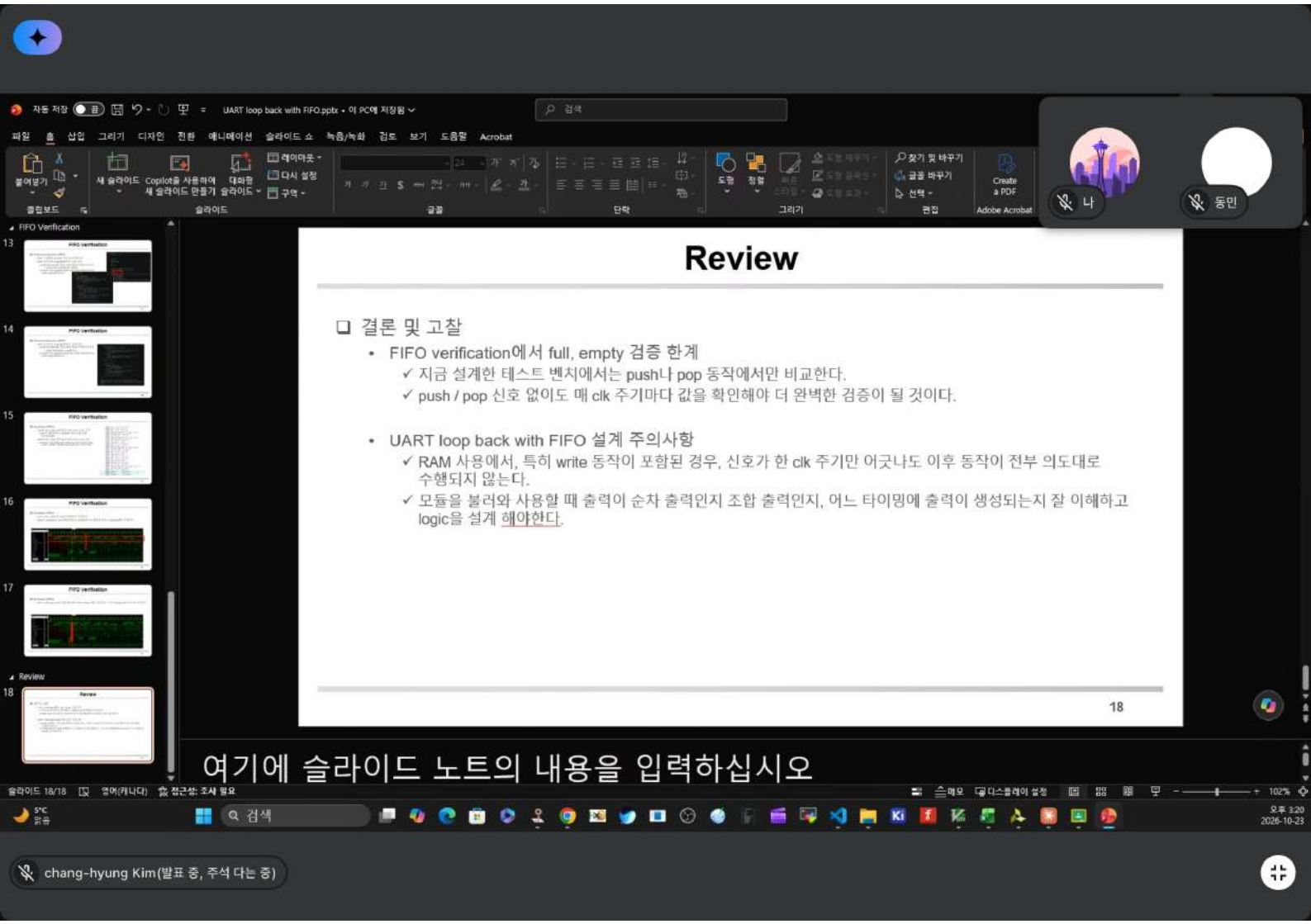
- Simulation (FIFO)
  - data와 full, empty 출력을 비교해 PASS, FAIL 판단
    - ✓ clk 주기에 판단하고, PASS와 FAIL의 수를 센 후 마지막에 출력
  - data와 full, empty 출력을 비교해 PASS, FAIL 판단
    - ✓ buffer가 pop일 때 push_cnt와 pop_cnt가 같으면 empty
      buffer가 push일 때 push_cnt와 pop_cnt가 같으면 full

```
10136000 : push = 1, pop = 0, i_data = 238
10146000 : push = 0, pop = 0, i_data = 22
10156000 : push = 0, pop = 1, i_data = 218
10160000 : PASS_data!! compare_data = 131, o_data = 131, pop = 1
10160000 : PASS_empty!! empty = 0, empty_buf = 0
10166000 : push = 0, pop = 1, i_data = 43
10170000 : PASS_data!! compare_data = 38, o_data = 38, pop = 1
10170000 : PASS_empty!! empty = 0, empty_buf = 0
10176000 : push = 1, pop = 0, i_data = 9
10180000 : compare_data = 9, push = 1
10180000 : PASS_full!! full = 0, full_buf = 0
10186000 : push = 0, pop = 1, i_data = 179
10190000 : PASS_data!! compare_data = 149, o_data = 149, pop = 1
10190000 : PASS_empty!! empty = 0, empty_buf = 0
10196000 : push = 0, pop = 0, i_data = 253
10206000 : push = 1, pop = 0, i_data = 212
10210000 : compare_data = 212, push = 1
10210000 : PASS_full!! full = 0, full_buf = 0
10216000 : push = 0, pop = 0, i_data = 67
10226000 : push = 1, pop = 0, i_data = 233
10230000 : compare_data = 233, push = 1
10230000 : PASS_full!! full = 0, full_buf = 0
10236000 : push = 0, pop = 1, i_data = 194
10240000 : PASS_data!! compare_data = 142, o_data = 142, pop = 1
10240000 : PASS_empty!! empty = 0, empty_buf = 0
10246000 : push = 0, pop = 0, i_data = 227
10250000 : PASS_data  =       447, FAIL_data  =       0
10250000 : PASS_full  =       450, FAIL_full  =       0
10250000 : PASS_empty =       447, FAIL_empty =       0
```

15

여기에 슬라이드 노트의 내용을 입력하십시오

chang-hyung Kim(발표 중, 주석 다는 중)


UART loop back with FIFO.pptx • 이 PC에 저장됨
나
동민
FIFO Verification
Review
❑ 결론 및 고찰
• FIFO verification에서 full, empty 검증 한계
✓ 지금 설계한 테스트 벤치에서는 push나 pop 동작에서만 비교한다.
✓ push / pop 신호 없이도 매 clk 주기마다 값을 확인해야 더 완벽한 검증이 될 것이다.
• UART loop back with FIFO 설계 주의사항
✓ RAM 사용에서, 특히 write 동작이 포함된 경우, 신호가 한 clk 주기만 어긋나도 이후 동작이 전부 의도대로 수행되지 않는다.
✓ 모듈을 불러와 사용할 때 출력이 순차 출력인지 조합 출력인지, 어느 타이밍에 출력이 생성되는지 잘 이해하고 logic을 설계 해야한다.
18
여기에 슬라이드 노트의 내용을 입력하십시오
chang-hyung Kim(발표 중, 주석 다는 중)